

```
$ find src-dir -type f -name "*.mp3" -exec mv {} target-dir \;
```

4) tar

Sometimes it is convenient to operate on a single file rather than multiple files and here, the *tar* command comes into the picture. *tar* is a short form for 'tape archive'. As the name suggests, it is an archiving utility that stores multiple files into a single one. Given below is the syntax of the *tar* command:

```
tar [OPTIONS] [TAR NAME] [FILES TO BE INCLUDED IN TAR]
```

To create a *tar* bundle, execute the command given below in a terminal:

```
$ tar cvf archive.tar 1.txt 2.txt 3.txt
```

In the above example:

- *c* option stands for create archive
- *v* option stands for verbose mode
- *f* option stands for file names mentioned for archive

The *tar* command allows us to manipulate *tar* bundles without recreating them again. For instance, to add a new file into an archive, use the *-r* option as shown below:

```
$ tar rvf archive.tar 4.txt
4.txt
```

```
$ tar tf archive.tar #list the content of tar file
1.txt
2.txt
3.txt
4.txt
```

By default, *tar* only archives multiple files; it doesn't do any compression. There are various compression utilities available like *bzip2*, *gzip*, *zip* and so on. To compress a *tar* bundle using *bzip2*, execute the command show below:

```
$ bzip2 archive.tar
```

After compression, it will append the *.bz2* extension to the *tar* bundle. If you compare sizes, before compression, the *tar* bundle was 20KB and after compression, it gets reduced to 4KB, as shown below:

```
# Size before compression
$ ls -sh archive.tar
20K archive.tar
```

```
# Size after compression
$ ls -sh archive.tar.bz2
4.0K archive.tar.bz2
```

5) fdisk

We partition disks for better management and utilisation of available storage. GNU/Linux provides the *gnome-disk* utility which is a GUI based application. However, we can do similar things with *fdisk*, which is a CLI based utility and can be used to manipulate the disk partition table.

Manipulating disk partitions recklessly will cause data loss; hence, we are going to use the *fdisk* command with a pseudo disk. We'll use a file as a disk, using the *losetup* command. Perform the steps given below to create a pseudo disk.

First, create a file of size 200MB using the *dd* command:

```
$ dd if=/dev/zero of=disk.img bs=1M count=200
```

Next, set up this file as a loop-back device so that, hereafter, we can use */dev/loop0* as a device:

```
$ sudo losetup /dev/loop0 disk.img
```

We can perform various actions with *fdisk* like printing the partition table, creating new partitions, deleting existing partitions, writing the partition table to disk, and so on. Let us perform all these actions, one by one.

To start the *fdisk* utility, use the command given below:

```
$ sudo fdisk /dev/loop0
```

After entering the above command, you will be shown a welcome message and the system will wait for a command to be entered. The section below describes various actions that can be performed using *fdisk*.

- **Print the partition table**
To print the partition table, type *p* and press *Enter*. This will display information about the disk and its partitions. As we haven't created any partition yet, it will show only information about the disk.
- **Create a new partition**
To create a new partition, type *n* and press *Enter*. Then follow the on-screen instructions.
- **Validate a created partition**
To view the created partitions, type *p* and press *Enter*.
- **Delete a partition**
To delete a partition, type *d* and press *Enter*. Follow the on-screen instructions to complete the procedure.
- **Write a partition table**
To make the changes permanent, we need to write this partition table to the disk. Type *w* and press *Enter* to complete this action.
- **Quit**
Type *q* and press *Enter* any time to quit the *fdisk* utility.

6) Networking related commands

Networking is an essential part of a computer system. However, it is complex and can be unstable sometimes. This